

LEMMON PRODUÇÕES

# Lemmon Agentes

Manual do Sistema

**Versão:** v1.2

**Data:** 2026-05-06

**Cliente principal:** Hator Clinic

Documento vivo · sempre que uma função for adicionada,  
este manual deve ser atualizado e um novo PDF gerado.

# LEMMON AGENTES — Manual do Sistema

---

**Versão atual:** v1.2 **Última atualização:** 2026-05-06 **Mantido por:** Calebe Alves / Lemmon Produções

Este é o documento de referência viva do sistema Lemmon Agentes. Sempre que uma função nova for implementada ou um épico fechar, este manual deve ser atualizado e uma nova versão de PDF gerada em [docs/releases/](#).

## Histórico de versões

---

**Convenção:** versões mais novas no topo. Cada release lista o que mudou em relação à anterior, mantendo histórico completo.

### v1.2 — 2026-05-06

#### Épico B — Pedro como gate de qualidade.

- **Gate espelho (T9):** Após o Salles e antes da Sônia, Pedro pode validar automaticamente se o roteiro está fiel à voz/posicionamento do cliente. Configurável por sessão no painel do Salles: `off` (padrão, sem gate), `auto` (bloqueia se veredicto ) , `manual` (sempre pede aprovação). O veredicto aparece no chat com badge //. Em modo `auto` , roteiros com  bloqueiam o pipeline e o operador decide se continua.
- **Mesa redonda stress test (T10):** Botão  `mesa` no toolbar de reunião. Escreva a tese no campo de input e clique para que cada agente presente questione a tese no seu ângulo específico (estratégia, compliance, roteiro, performance, cliente espelho). Aya sintetiza uma ata executiva. Endpoint: `GET /ws/mesa_redonda` .

### v1.1 — 2026-05-06

#### Épico A — Família de espelhos de cliente.

- **EspelhoCliente (T6):** classe genérica extraída de `PedroAbrahao` . Qualquer cliente espelho é agora uma instância paramétrica de `EspelhoCliente(id, nome, material_dir, ...)` . Pedro virou uma factory function em `agentes/pedro_abrahao.py` . Material primário migrado para `inputs/clientes/pedro/` .
- **Wizard de onboarding (T7):** `onboard_cliente.py` — CLI interativo (ou com args `--id --nome --nicho --cor` ) que cria automaticamente dossiê, transcrições, system prompt e pasta de outputs para um novo cliente espelho. Também gera snippet TypeScript para `agents.ts` e exemplo de instanciação Python.

- **Salas temáticas (T8):** sala de reunião muda paleta visual (tapete, brilhos) conforme o cliente ativo. Seletor de cliente no header da cena — exibe nome + cor do cliente; botão "trocar 🔄" aparece quando há mais de um cliente registrado. Para adicionar tema de novo cliente: incluir entrada em `MEETING_THEMES` em `OfficeScene.tsx`.

**Para adicionar um novo cliente espelho:**

```
python onboard_cliente.py # wizard interativo
# ou com args:
python onboard_cliente.py --id marina --nome "Marina Costa" --nicho "nutricionista"
```

Depois preencher `inputs/clientes/<id>/dossie.md`, colar transcrições reais, e adicionar o snippet TypeScript gerado em `dashboard/lib/agents.ts` e a entrada de tema em `dashboard/components/office/OfficeScene.tsx > MEETING_THEMES`.

---

## v1.0 — 2026-05-05

**Primeira versão publicada.** Documenta o estado atual do sistema:

- 6 agentes operacionais: Otto, Heitor, Salles, Sônia, Aya, Pedro
- Pipeline sequencial e Modo Reunião conversacional
- Modo Manual (aprovação step-by-step)
- Retomada de sessão (continuar trabalho anterior)
- Histórico persistido + avaliação por estrelas
- Upload de imagem com descrição automática (visão Haiku)
- Speech-to-text no input
- Streaming nativo da Anthropic em modo Reunião
- URL backend centralizada ( `API_URL` / `WS_URL` )
- 6 CLIs individuais + pipeline completo via terminal

**Correções incorporadas (T1–T5 do PLANO\_ACAO):** custo do Otto padronizado, Aya recebe contexto técnico, `/avaliar` lê do disco, URL centralizada, streaming nativo em Reunião.

---

## Sumário

1. Visão geral
2. A equipe — 6 agentes
3. Como o sistema funciona — modos de operação
4. Funcionalidades do dashboard
5. CLI direto (terminal)
6. Receitas — workflows recomendados
7. Roadmap — o que está vindo
8. Apêndice — custos, limites, configuração
9. Como atualizar este manual



# 1. Visão geral

---

O **Lemmon Agentes** é o sistema interno de produção de conteúdo da Lemmon Produções. Em vez de Calebe escrever roteiros do zero a cada projeto, ele convoca uma equipe de agentes especializados que processam o briefing em camadas: estratégia, compliance, roteiro, performance, compilação, e validação por espelho de cliente.

O sistema funciona em dois modos principais. O **Pipeline** corre em sequência fixa e é otimizado para entrega rápida de uma sessão completa. O **Modo Reunião** é conversacional, multi-turno, e permite discussão livre entre os agentes com menções estilo Slack. Os dois usam a mesma equipe de fundo, mas com posturas diferentes.

Tecnicamente o sistema é composto por: backend Python (FastAPI + WebSocket) que orquestra os agentes via API da Anthropic; dashboard Next.js/React com escritório virtual onde os personagens vivem; e histórico persistido em JSON local para auditoria, avaliação e retomada de sessões. Tudo roda no computador do operador — sem cloud, sem servidor.

**Filosofia:** o sistema não substitui o operador. Ele acelera o pensamento ao dar uma equipe de cabeças especializadas trabalhando em paralelo. Calebe continua sendo o diretor — os agentes são os redatores assistentes.

## 2. A equipe — 6 agentes

A equipe atual tem 6 personagens. Cinco entram no pipeline padrão (Otto → Heitor → Salles → Sônia → Aya); o sexto, Pedro, é convocado sob demanda em modo Reunião.

### 2.1 Otto — Estrategista

**Cor:** azul-marinho · **Classe RPG:** Analista

**Papel.** Otto é o primeiro a tocar o briefing. Ele decodifica o que o cliente pediu, o que o cliente NÃO disse, identifica o conflito central, a insegurança subjacente, e produz uma tese criativa. Saída técnica em formato estruturado (tool use forçado), mais uma versão humana em markdown.

**Quando usar.** Sempre que houver briefing novo. Otto é o ponto de partida do pipeline. Em modo Reunião ele entra quando você quer um olhar estratégico sobre uma ideia ou validação de tese.

**Configurações.**

| Parâmetro                | Valores                                                                | Descrição                                                                          |
|--------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <code>modo_visual</code> | <code>completo</code> , <code>resumido</code> ,<br><code>mínimo</code> | Tamanho do output. Completo dá tudo; resumido só tese + conceito; mínimo é tese só |

**Exemplo de input.** "Cliente é uma clínica de medicina personalizada para mulheres 35-55. Quer um vídeo curto sobre tratamento de menopausa que não soe como propaganda."

**Exemplo de output (resumido).**

TESE: a verdadeira reposição é de identidade, não de hormônio.

CONCEITO: "A mulher que você vai voltar a ser" — formato testemunhal, posicionamento da clínica como restauradora de algo perdido, não vendedora de tratamento.

**Custo médio.** \$0.02–0.08 por execução (briefing simples a complexo).

### 2.2 Heitor — Compliance

**Cor:** verde-musgo · **Classe RPG:** Guardião

**Papel.** Heitor é a barreira contra falar bobagem. Verifica termos críticos contra Anvisa, conselhos profissionais (CFM, CRM, CFO, CRO, CFN, CFF, CFP, COFFITO), e diretrizes da Meta. Pode buscar em domínios oficiais para checar se uma reivindicação se sustenta.

**Quando usar.** Para qualquer conteúdo que vá ao público em saúde, beleza, suplementação, terapias. Em pipeline interno (proposta, pitch da Lemmon) Heitor pode ser pulado.

**Configurações.**

| Parâmetro                | Valores | Descrição                                                     |
|--------------------------|---------|---------------------------------------------------------------|
| <code>max_buscas</code>  | 1–10    | Quantas buscas web Heitor pode fazer (padrão 3, profundo 6)   |
| <code>secundarias</code> | bool    | Permite buscar em fontes não-oficiais (jornalismo, marketing) |

**Saída.** Risco geral verde / amarelo / vermelho. Lista de termos críticos identificados. Sugestões de reescrita. URLs das fontes consultadas.

**Comportamento de confirmação.** Antes de buscas longas pede confirmação ao operador via WebSocket (callback `_make_confirmacao_callback`). Em modo manual o operador aprova cada busca; em modo automático com aviso de custo acima do threshold (\$0.50), pede confirmação.

**Custo médio.** \$0.20–0.40 por execução com buscas. Sem buscas, \$0.05–0.10.

## 2.3 Salles — Roteirista

**Cor:** terracota · **Classe RPG:** Criativo

**Papel.** Transforma a tese do Otto em roteiro filmável. Trabalha com formatos pré-definidos. Sabe questionar a estratégia (ver `core/discussao.py` — Salles pode rodar uma rodada de discussão com o Otto antes de escrever, contestando viabilidade técnica e foco narrativo).

**Quando usar.** Sempre que a saída final for um roteiro a ser produzido. Em validação de pitch ou apresentação institucional, Salles é dispensável.

**Configurações.**

| Parâmetro            | Valores                                                                                                                                |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <code>formato</code> | <code>auto</code> , <code>reels</code> , <code>documental</code> , <code>mini-doc</code> , <code>tese</code> , <code>aftermovie</code> |

**Saída.** Roteiro em markdown com bloco-a-bloco, indicação de ação, fala, b-roll sugerido. Estrutura técnica em JSON paralelo (`formato_aplicado`, `num_blocos`, `titulo_roteiro`).

**Custo médio.** \$0.10–0.25 por roteiro.

## 2.4 Sônia — Performance

**Cor:** violeta · **Classe RPG:** Growth

**Papel.** Avalia o roteiro do ponto de vista de performance: hooks, retenção, CTA, adequação a tendências. Pode buscar tendências atuais em domínios pré-aprovados (Meta business, criadores oficiais). Sugere cortes autônomos a partir do roteiro longo.

**Quando usar.** Após Salles em qualquer projeto que vá para distribuição em rede social. Em peças institucionais offline, dispensável.

## Configurações.

| Parâmetro                    | Valores                                                                 | Descrição                                                                           |
|------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>com_busca</code>       | <code>bool</code>                                                       | Ativa web search (custo extra)                                                      |
| <code>usar_tendencias</code> | <code>bool</code>                                                       | Injeta <code>inputs/tendencias_atuais.md</code> no prompt                           |
| <code>modo</code>            | <code>cadeia</code> , <code>solo</code> ,<br><code>cortes_apenas</code> | Cadeia = passa por análise master; solo = direto;<br>cortes_apenas = só gera cortes |

**Saída.** Nota master 0–10. Análise por critério. Lista de cortes autônomos com timestamps sugeridos. Peça destaque indicada.

**Custo médio.** \$0.15–0.25 sem busca; \$0.30–0.50 com busca.

## 2.5 Aya — Compiladora (Oráculo)

**Cor:** preto · **Classe RPG:** Oráculo

**Papel.** Compila os outputs dos outros agentes em um dossiê único, organizado, em markdown.

**Não interpreta, não sintetiza, não opina** — só organiza. Arquitetura em duas etapas: chamada API que produz cards de resumo + montagem Python pura do markdown final.

**Quando usar.** Como última etapa do pipeline para entregar um deliverable consolidado. Também em modo Reunião quando você quer perguntar coisas sobre o sistema ou a equipe (em reunião usa `system_prompt_reuniao` reduzido, sem material de compilação).

**Configurações.** Aya não tem configurações no dashboard hoje. Pode receber `outputs_diretos` (no pipeline já é passado automaticamente desde T2) ou `arquivos_especificos` (CLI).

**Saída.** Dossiê markdown com cabeçalho, índice, página de resumo dos agentes, e seções completas de cada agente que rodou. **Regra de ouro: agentes ausentes não aparecem no dossiê.**

**Custo médio.** \$0.05–0.20.

## 2.6 Pedro — Consultor (Cliente Hator)

**Cor:** verde-azulado ( `#0f766e` ) · **Classe RPG:** Cliente

**Papel.** Espelho IA do Dr. Pedro Abrahão (Hator Clinic). Treinado com dossiê de posicionamento + transcrições reais. Avalia conteúdo da ótica do cliente: voz fiel? posicionamento correto? cacoetes verbais respeitados? Recusa zonas (diagnóstico médico, decisões grandes do negócio, temas íntimos).

**Quando usar.** Sob demanda em modo Reunião — ele tem flag `reuniaoOnly: true` e não entra no pipeline padrão. Útil para validar roteiro pronto antes de gravação, simular reação do cliente a uma proposta, ou perguntar "como Pedro responderia a X?".

**Configurações.** No CLI ( `pedro_cli.py` ):

| Parâmetro               | Valores                                                                           |
|-------------------------|-----------------------------------------------------------------------------------|
| <code>--modo</code>     | <code>validacao</code> , <code>consulta</code> , <code>resposta_hipotetica</code> |
| <code>--contexto</code> | Caminho de arquivo com texto a avaliar (ex: roteiro pronto)                       |

**Material primário.** Carregado via `EspelhoCliente._carregar_material_primario()` a partir de `inputs/clientes/pedro/dossie.md` + `inputs/clientes/pedro/transcricoes.md`. Injetado em `system_prompt_reuniao` — funciona corretamente em modo Reunião e CLI.

#### Como instanciar (Python):

```
from agentes.pedro_abrahaao import PedroAbrahaao
pedro = PedroAbrahaao()
resultado = pedro.executar(pergunta="...", modo="validacao")
```

**Custo médio.** \$0.05–0.20.

## 3. Como o sistema funciona — modos de operação

---

### 3.1 Modo Pipeline

---

Sequência fixa: Otto → Heitor → Salles → Sônia → Aya. Sem volta, sem ramificação. Cada agente recebe o output do anterior, produz o seu, passa adiante. Aya é sempre a última e compila tudo.

**Quando usar.** Briefing novo, deliverable claro, deadline. Quando você sabe o que quer e só precisa que o sistema entregue.

**Como ativar.** No dashboard, clique nos agentes que devem entrar no pipeline (cards no header). Modo "pipeline" no toggle do chat panel. Envie o briefing.

**Comportamento.** WebSocket `/ws/chat` recebe `agents`, `message`, `manual_mode`, `config`. Para cada agente em ordem, executa, faz streaming dos tokens, salva resposta. Ao final, salva sessão completa em `historico/dashboard/<timestamp>_sessao.json` e envia `pipeline_done` com `session_id`.

### 3.2 Modo Reunião

---

Conversacional, multi-turno. Você manda mensagem, agentes respondem (em ordem). Próxima mensagem entra como turno seguinte. Histórico persistido client-side (sobrevive reconexão WS) e server-side (salvo a cada turno em `historico/dashboard/<timestamp>_reuniao.json`).

**Quando usar.** Brainstorm, validação de ideia, debate de direção criativa, discussão entre agentes. Quando você não sabe ainda o que quer, ou quer ouvir várias cabeças antes de decidir.

**Como ativar.** Toggle "conv." no chat panel. Convoque os agentes que devem participar.

**Menções (@).** Em modo manual da reunião, só agentes mencionados respondem. Em modo auto, todos respondem em ordem (a menos que um seja mencionado especificamente). Use `@otto`, `@heitor`, etc.

**Streaming.** Modo Reunião usa streaming nativo da Anthropic (token a token, real). Pipeline ainda usa stream simulado por questão de tool use forçado nos agentes.

### 3.3 Modo Manual (aprovação step-by-step)

---

Toggle no header do chat panel. Em vez de o pipeline correr direto até o fim, o sistema pausa após cada agente e espera o operador clicar **Continuar**, **Pular**, **Tentar de novo** ou **Cancelar**.

**Quando usar.** Em projetos sensíveis ou caros onde você quer ver o output de cada etapa antes de seguir. Também útil para depurar comportamento estranho de um agente.

**Aceite por step.** Em caso de erro do agente em modo manual, três opções aparecem na barra de aprovação: **Retry** (tenta de novo), **Pular** (segue sem o output desse agente) ou **Cancelar** (encerra pipeline).

### 3.4 Retomada de sessão

---

Você pode pegar uma sessão antiga do histórico, clicar **Retomar**, e o sistema recarrega contexto técnico (análise Otto, diretrizes Heitor, roteiro Salles, etc.) e abre o chat para você continuar.

**Quando usar.** Cliente voltou com ajuste 3 dias depois. Você quer pegar a tese do Otto e rodar Salles num formato diferente. Você quer só rodar Aya numa sessão antiga que não tinha sido compilada.

**Como.** Histórico → escolher sessão de pipeline (não funciona com reunião por enquanto) → botão **Retomar**. Banner laranja aparece no chat: "Sessão retomada — selecione agentes e continue". Selecione apenas os agentes que precisam re-rodar e mande o ajuste como nova mensagem.

**Detalhe.** Ao retomar, se você mandar nova mensagem, ela vira [INSTRUÇÃO ADICIONAL] concatenada ao briefing original. Se mandar mensagem vazia (placeholder), o sistema usa só o briefing original.

### 3.5 Avaliação

---

Após cada sessão concluída, aparece barra "Como foi essa sessão?" com 5 estrelas. Avaliação salva no JSON da sessão. Pode também adicionar observações e tags.

**Quando usar.** Sempre que possível — vira combustível para o sistema aprender (Épico C do plano: few-shot curado de 5★).

**Onde fica.** No JSON da sessão como `avaliacao`, `observacoes_operador`, `tags`. Endpoint `/avaliar` lê do disco (sobrevive a reinício do servidor).

---

## 4. Funcionalidades do dashboard

---

### 4.1 Upload de imagem

---

Clipe no canto inferior esquerdo do input. Imagem (JPG/PNG/WEBP, máx 5MB) é descrita automaticamente por Claude Haiku 4.5 e a descrição é injetada no briefing antes de chegar nos agentes. Útil para:

- Foto da clínica → contexto visual entra na análise do Otto
- Print de tendência do Instagram → Sônia pode reagir
- Mood board → Salles pega o tom

**Falha silenciosa atualmente.** Se a descrição da imagem falha (rate limit, etc.), o briefing segue sem o contexto visual e o operador não é avisado. Polimento C do plano corrige isso.

### 4.2 Speech-to-text

---

Microfone no canto direito do input. Usa Web Speech API ( `SpeechRecognition` ) em pt-BR. Continuous + interim results. Útil para gravar briefing em voz quando você não quer digitar.

**Limitação.** Funciona melhor no Chrome/Edge. Safari pode falhar. Firefox não tem.

### 4.3 Exportar sessão

---

Botão de download no header do chat panel. Gera arquivo `lemmon_sessao_<data>.txt` com todas as mensagens da sessão atual, incluindo custos por agente. Útil para arquivar fora do sistema, mandar pra cliente, anexar em email.

### 4.4 Histórico

---

Painel flutuante, ícone de relógio no header geral. Lista as 200 sessões mais recentes (limite atual). Cada item mostra: timestamp, briefing truncado, agentes usados, custo total, avaliação, e badge se é pipeline ou reunião.

**Detalhe da sessão.** Click → painel abre detalhe completo, incluindo respostas por agente, custos individuais, e — para reuniões — o histórico cronológico dos turnos.

**Filtros.** Hoje não há filtros (planejado em T17 do Épico C).

## 4.5 Painéis flutuantes

---

Tanto o ChatPanel quanto o HistoryPanel são arrastáveis (drag pelo header) e redimensionáveis (handles nas bordas). Posições persistem na sessão atual; resetam ao recarregar.

## 4.6 Escritório virtual

---

Cena RPG com sprites dos agentes em mesas. Quando você convoca um agente, ele caminha da mesa para a sala de reunião. Status físico (idle, thinking, speaking, done, error) reflete em cor e animação. Idle quotes aparecem em bolas de fala periodicamente.

**Roadmap:** os Épicos G do plano aprofundam a camada visual — whiteboards que se preenchem em tempo real, mic destacado em quem fala, salas customizadas por cliente.

## 5. CLI direto (terminal)

---

Cada agente tem um CLI próprio para uso fora do dashboard. Útil para automação, scripts, debugging.

### 5.1 Otto

---

```
python otto_cli.py inputs/briefing.txt
python otto_cli.py inputs/briefing.txt --modo-visual resumido
```

### 5.2 Heitor

---

```
python heitor_cli.py inputs/conteudo.md --max-buscas 5
python heitor_cli.py inputs/conteudo.md --no-confirm
```

### 5.3 Salles

---

```
python salles_cli.py inputs/briefing.txt --formato reels
python salles_cli.py inputs/briefing.txt --formato mini-doc --tags hator,menopausa
```

### 5.4 Sônia

---

```
python sonia_cli.py outputs/salles/<roteiro>.md
python sonia_cli.py outputs/salles/<roteiro>.md --com-busca --modo cadeia
```

### 5.5 Aya

---

```
python aya_cli.py
python aya_cli.py --nome-projeto "Hator menopausa"
```

## 5.6 Pedro

---

```
python pedro_cli.py "como você responderia se uma paciente perguntasse X"  
python pedro_cli.py inputs/pergunta.txt --modo validacao --contexto outputs/salles/roteiro.md
```

## 5.7 Pipeline completo

---

```
python pipeline_completo.py inputs/briefing.txt  
python pipeline_completo.py inputs/briefing.txt --formato reels --com-aya  
python pipeline_completo.py inputs/briefing.txt --profundo --busca-sonia --com-aya
```

Flags úteis: `--sem-heitor`, `--sem-sonia`, `--no-confirm`, `--profundo`, `--sonia-profundo`, `--nome-projeto "X"`.

---

## 6. Receitas — workflows recomendados

---

### 6.1 Roteiro novo do zero (cliente Hator)

---

1. Abrir dashboard
2. Convocar Otto, Heitor, Salles, Sônia, Aya
3. Modo pipeline, modo manual ativo (recomendado)
4. Enviar briefing
5. Aprovar Otto após ler tese
6. Aprovar Heitor (se risco vermelho, decidir continuar ou parar)
7. Aprovar Salles após ler roteiro
8. Aprovar Sônia
9. Aya compila automaticamente
10. Avaliar sessão (5★ se ficou bom)

**Tempo médio:** 5–10 min com modo manual, 2–4 min em automático.

### 6.2 Validação de roteiro pronto

---

Você já tem um roteiro pronto e quer validação antes de gravar.

1. Modo Reunião
2. Convocar Pedro + Heitor (gate de cliente + gate de compliance)
3. Mensagem: @pedro avalie esse roteiro: [colar roteiro]
4. Pedro responde com observações de fidelidade
5. Mensagem: @heitor passa por compliance: [colar trecho de risco]
6. Decidir ajustes baseado nos dois pareceres

### 6.3 Brainstorm de tese

---

Você não tem briefing fechado ainda. Quer pensar em voz alta com a equipe.

1. Modo Reunião
2. Convocar Otto + Salles + Sônia
3. Modo manual da reunião (só responde se @mencionado)
4. Conversar livremente, mencionar quem deve opinar quando

## 6.4 Variação de algo que já funcionou

---

1. Histórico → encontrar sessão antiga 5★
2. Botão **Retomar**
3. Modo pipeline com Salles + Sônia + Aya (Otto não precisa rodar de novo)
4. Mensagem: mesma tese, formato Reels em vez de mini-doc
5. Pipeline herda análise estratégica antiga e regenera só o que mudou

## 6.5 Checagem só de compliance

---

Você tem um post pronto e só quer ver se passa pelo Heitor.

1. CLI: `python heitor_cli.py inputs/post.md`
2. Ler relatório de risco e termos críticos
3. Ajustar texto e rodar de novo se necessário

## 6.6 Rodar em background (sem dashboard)

---

Para automação ou processamento em lote, use o `pipeline_completo.py`:

```
for briefing in inputs/lote/*.txt; do
    python pipeline_completo.py "$briefing" --com-aya --no-confirm
done
```

## 7. Roadmap

---

O `PLANO_ACAO_2026-05-05.md` na raiz do projeto contém o plano completo com 9 épicos e 39 tarefas. Resumo do que está no horizonte:

**Próximos passos imediatos.** ~~Família de espelhos de cliente~~  (v1.1), ~~Pedro como gate de qualidade~~  (v1.2), Pulse semanal automatizado (relatório institucional toda segunda).

**Médio prazo.** Novos agentes (Marcia/pós-produção, Felipe/concorrente, Renata/distribuição, Lia/voz Lemmon). Workflows novos (modo remix, briefing reverso, comparativo A/B no Salles, cortes-prontos).

**Aprendizado.** Few-shot curado de sessões 5★, busca semântica no histórico, Hall of Fame compartilhável, tags semi-automáticas, dashboard de saúde do sistema.

**Camada visual.** Whiteboards que se preenchem em tempo real, sprites com status físico mais expressivo, mesa de reunião com mic destacado, salas customizáveis por cliente.

**Multimodal.** Memo de voz como briefing (upload de áudio), output como áudio (TTS), link de aprovação compartilhável com cliente.

Cada épico fechado deve atualizar este manual e gerar nova versão de PDF em `docs/releases/`.

## 8. Apêndice — custos, limites, configuração

### 8.1 Faixas de custo previstas

Configurado em `core/config.py` :

| Agente             | Faixa esperada por execução |
|--------------------|-----------------------------|
| Otto               | \$0.02–0.08                 |
| Heitor (sem busca) | \$0.05–0.10                 |
| Heitor (com busca) | \$0.20–0.40                 |
| Salles             | \$0.10–0.25                 |
| Sônia (sem busca)  | \$0.15–0.25                 |
| Sônia (com busca)  | \$0.30–0.50                 |
| Aya                | \$0.05–0.20                 |
| Pedro              | \$0.05–0.20                 |

**Custo total típico de pipeline completo:** \$0.50–1.50.

**Threshold de alerta de pipeline caro:** \$1.00 ( `PIPELINE_AVISO_CUSTO_TOTAL_USD` ).

### 8.2 Limites de tamanho

|                                          |   |                    |                                         |   |                     |
|------------------------------------------|---|--------------------|-----------------------------------------|---|---------------------|
| <code>BRIEFING_MIN_CARACTERES</code>     | = | <code>50</code>    | <code>BRIEFING_MAX_CARACTERES</code>    | = | <code>15000</code>  |
| <code>AYA_OUTPUT_AGENTE_MAX_CHARS</code> | = | <code>15000</code> | <code>AYA_DOSSIE_MAX_CHARS_TOTAL</code> | = | <code>100000</code> |
| <code>PEDRO_INPUT_MAX_CHARS</code>       | = | <code>20000</code> | <code>SONIA_ROTEIRO_MAX_CHARS</code>    | = | <code>30000</code>  |

### 8.3 Modelo padrão

`claude-sonnet-4-6` configurado em `LEMMON_MODELO_PADRAO` (env var) ou `core/config.py` .

Visão (descrição de imagem upload): `claude-haiku-4-5-20251001` .

## 8.4 Estrutura de pastas

```
lemmon-agentes/
├── agentes/          # implementações dos 6 agentes
├── core/             # base, custos, validador, espelho genérico
│   ├── espelho.py    # EspelhoCliente – classe genérica de espelho de cliente
│   └── limites_espelho.py # avisos de custo pré/pós execução para espelhos
├── prompts/         # system prompts versionados (v1, v2, v3)
├── inputs/          # briefings, dossiês, transcrições
│   └── clientes/     # material primário por cliente espelho
│       └── pedro/    # dossie.md + transcricoes.md do Dr. Pedro Abrahão
├── outputs/         # outputs de execuções (por agente e por cliente)
├── historico/       # sessões salvas (JSON)
├── dashboard/       # frontend Next.js
├── docs/            # este manual e releases
├── onboard_cliente.py # wizard CLI para novo cliente espelho
└── PLANO_ACAO_*.md   # plano de implementação
```

## 8.5 Variáveis de ambiente

| Var                  | Default               | Função                 |
|----------------------|-----------------------|------------------------|
| ANTHROPIC_API_KEY    | (obrigatório)         | Chave da API           |
| LEMMON_MODELO_PADRAO | claude-sonnet-4-6     | Modelo dos agentes     |
| LEMMON_LOG_LEVEL     | INFO                  | Nível de log           |
| NEXT_PUBLIC_API_URL  | http://localhost:8000 | URL backend (frontend) |

## 8.6 Como subir o sistema localmente

Backend:

```
cd lemmon-agentes
source .venv/bin/activate # ou criar com python -m venv .venv
pip install -r requirements.txt # se existir
uvicorn api_server:app --reload --port 8000
```

Frontend:

```
cd lemmon-agentes/dashboard
npm install
npm run dev # http://localhost:3000
```

## 9. Como atualizar este manual

---

### 9.1 Quando atualizar

---

- Sempre que uma tarefa do PLANO\_ACAO for fechada
- Quando uma função nova for adicionada
- Quando um agente for criado, alterado significativamente, ou removido
- Quando comportamento documentado aqui mudar

### 9.2 Como atualizar

---

1. **Editar** `MANUAL_SISTEMA.md` — fazer as mudanças no markdown. Esta é a fonte de verdade.
2. **Atualizar cabeçalho:**
3. `**Versão atual:** vX.Y` — incrementar (v1.0 → v1.1 para mudanças menores; v1.0 → v2.0 para reformulações)
4. `**Última atualização:** YYYY-MM-DD`
5. **Adicionar entrada no** `## Histórico de versões` (no topo da seção, novidades sempre primeiro): ``` ### v1.1 — 2026-05-15`

**O que mudou:** - Adicionado agente Marcia (pós-produção) - Modo remix documentado em §6  
``

1. **Atualizar** `CHANGELOG.md` com entrada equivalente.
2. **Gerar PDF:** `bash cd /Users/calebe/Documents/lemmon-agentes python docs/gerar_pdf.py`  
Saída: `docs/releases/MANUAL_v<versao>_<YYYY-MM-DD>.pdf`

### 9.3 Convenção de versionamento

---

**Major (v2.0).** Reformulação grande do sistema. Mudança de paradigma. Renomeação de agentes principais. Quebra de compatibilidade com sessões antigas.

**Minor (v1.1, v1.2).** Funções novas. Agentes novos. Modos de operação novos.

**Patch (v1.0.1).** Correção de doc, ajuste fino de descrição, sem mudança real no sistema.

## 9.4 Releases nunca somem

---

Cada PDF gerado em `docs/releases/` permanece para sempre como snapshot histórico. **Não apagar.** Útil para:

- Auditoria do estado do sistema em data X
  - Comparação de evolução
  - Onboarding de pessoa nova ("o sistema em v1.0 era assim, agora está em v1.5")
  - Documentação para cliente externo (Hator pode receber v atual sem ver a próxima ainda em desenvolvimento)
- 

Manual mantido como documento vivo · Lemmon Produções · 2026